

Go CI 单元测试与 GitHub PR 自动合并

生产实践与 GitHub 配置操作手册

适用场景：Go 项目 • GitHub Hosted Runner • Required Checks • GitHub Auto-merge

目标：PR 创建后自动跑 CI，检查全部通过后自动合并到 main

文档定位

本手册把“Go 单元测试”和“PR 自动合并”合并为一套可落地方案，并把所有涉及 GitHub UI 的配置路径逐步写明。现有 AI 安全审计可继续作为第二个 Required Check，本手册不重复展开其模型与 Skill 实现细节。

1. 目标架构与最终效果

完成配置后，一个标准功能分支的生命周期如下：



1.1 推荐仓库结构

```
project/
├── go.mod
├── go.sum
├── cmd/
└── internal/
```

```
└─ pkg/
  └─ .github/
    └─ workflows/
      └─ test.yml
      └─ audit.yml
      └─ auto-merge.yml # 已有 AI 安全审计时保留
```

1.2 三个 Workflow 的职责边界

文件	职责	是否需要写权限
test.yml	Go 单元测试；PR 更新时重新测试	否，contents: read
audit.yml	AI/静态安全审计；形成 Security Audit Check	通常否，contents: read
auto-merge.yml	给当前 PR 开启 GitHub Auto-merge	是，pull-requests/contents: write

2. Go 单元测试 Workflow

建议先创建并跑通 test.yml，因为后续 Ruleset 需要把这个 Job 产生的 Check 设置为 Required Status Check。

2.1 创建文件

- 1. 在项目根目录创建目录 .github/workflows（如果已经存在则跳过）。
- 2. 创建文件 .github/workflows/test.yml。
- 3. 复制下面的生产基础版本。

```
name: Go Unit Tests

on:
  pull_request:
    branches:
      - main
    types:
      - opened
      - synchronize
      - reopened

  # 手工合并、普通 push main 时可再次验证主干。
  # 关于 GITHUB_TOKEN 自动合并后的触发注意事项见第 8 章。
  push:
    branches:
      - main

  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: unit-tests-${{ github.event.pull_request.number || github.ref }}
```

```
cancel-in-progress: true

jobs:
  unit-test:
    name: Go Unit Tests
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: Checkout code
        uses: actions/checkout@v6

      - name: Setup Go
        uses: actions/setup-go@v7
        with:
          go-version-file: go.mod
          cache: true

      - name: Show Go version
        run: go version

      - name: Download dependencies
        run: go mod download

      - name: Run unit tests
        run: go test -v -count=1 ./...
```

2.2 关键字段说明

配置	作用
pull_request → main	创建 PR、PR 追加 commit、重新打开 PR 时执行测试。
push → main	普通 push/人工合并后再次验证 main；自动合并的特殊情况见第 8 章。
workflow_dispatch	允许在 GitHub Actions 页面手动执行。
permissions: contents: read	单元测试只读代码，遵循最小权限。
concurrency	同一 PR 有新 commit 时取消旧测试，只保留最新代码的结果。
go-version-file: go.mod	直接读取 go.mod 中的 Go 版本，避免 CI 与项目版本漂移。
go test -v -count=1 ./...	执行所有 package 测试，并避免直接复用测试结果缓存。

Required Check 名称

Ruleset 中需要添加的状态检查名称是 Job 的 name，也就是 “Go Unit Tests”。请保持该名称稳定，不要频繁修改，否则 Ruleset 里的 Required Check 需要同步调整。

3. 第一次提交并确认 Unit Test Check

3.1 提交 Workflow

```
git add .github/workflows/test.yml
git commit -m "ci: add Go unit tests"
git push origin <your-feature-branch>
```

3.2 创建 PR

如果本地安装了 GitHub CLI (gh)，可直接执行：

```
gh pr create --base main --fill
```

如果尚未安装 gh，也可以在 GitHub 网页中创建 Pull Request。

3.3 在 GitHub UI 检查执行结果

1. GitHub → 打开目标 Repository。
2. Repository 顶部菜单 → Actions。
3. 左侧选择 “Go Unit Tests”。
4. 打开刚刚由 PR 触发的 Workflow Run。
5. 确认 Job “Go Unit Tests” 为绿色 Success。
6. 再进入 Pull requests → 打开该 PR → Checks，确认能看到 “Go Unit Tests”。

为什么要先跑一次？

Required status check 必须是仓库中实际存在的检查。生产配置前先让 “Go Unit Tests” 成功执行一次，后续在 Ruleset 中添加它会更直观，也便于确认名称完全一致。

4. GitHub Repository 必做配置

自动合并是否可靠，关键不在 auto-merge.yml 本身，而在 Repository 的 Ruleset 和 Auto-merge 设置。以下步骤按当前 GitHub Repository UI 编排。

4.1 开启仓库 Auto-merge

1. GitHub → Repository。
2. Repository 顶部 → Settings。

- 3. 左侧菜单 → General。
- 4. 向下滚动到 “Pull Requests” 区域。
- 5. 勾选 “Allow auto-merge” 。
- 6. 建议同时确认允许的 Merge 方法：如团队统一使用 Squash，勾选 “Allow squash merging” 。

Auto-merge 的作用

开启后，PR 可以进入 “等待合并” 状态。当 Required Reviews（如果配置）和 Required Status Checks 全部满足时，GitHub 自动执行合并。

5. 为 main 创建 Branch Ruleset

推荐使用 GitHub Rulesets，而不是只依赖旧版 Branch protection rule。Ruleset 可以集中定义 PR、状态检查和分支保护规则。

5.1 创建 Ruleset 的 UI 路径

- 1. GitHub → Repository。
- 2. Repository 顶部 → Settings。
- 3. 左侧 “Code and automation” 区域 → Rules。
- 4. Rules → Rulesets。
- 5. 点击 “New ruleset” 。
- 6. 选择 “New branch ruleset” 。

5.2 基础信息

配置项	推荐值	说明
Ruleset name	main-protection	名称清晰即可。
Enforcement status	Active	正式启用并强制执行。
Bypass list	尽量为空	避免管理员/机器人绕过 CI；确有应急需求再单独授权。

5.3 Target branches：只保护 main

- 1. 在 Ruleset 的 “Target branches” 区域点击 “Add target” 。

- 2. 如果 main 是默认分支，可选择 “Include default branch”。
- 3. 或者选择按名称/模式包含分支，并指定 main。
- 4. 确认规则目标只覆盖你希望保护的分支；本方案至少覆盖 main。

5.4 开启 Require a pull request before merging

- 1. 向下找到 “Branch protections”。
- 2. 勾选 “Require a pull request before merging”。
- 3. 如果目标是 “CI 全绿后完全自动合并”，不要额外要求人工 Approval。
- 4. 如果团队仍要求人工 Review，则配置 Required approvals（例如 1）；此时 Auto-merge 会继续等待 Review 通过后再合并。

自动 ≠ 绕过审批

GitHub Auto-merge 不会绕过 Ruleset。Ruleset 要求什么，Auto-merge 就等待什么。需要完全无人值守就不要开启人工 Approval；需要人工把关就保留 Approval。

5.5 开启 Require status checks to pass before merging

- 1. 在同一 Ruleset 中勾选 “Require status checks to pass before merging”。
- 2. 点击添加 Status Check。
- 3. 加入 “Go Unit Tests”。
- 4. 如果当前项目已经启用 AI 安全审计，再加入 “GLM-5.2 Security Audit” 或你实际 audit Job 的固定名称。
- 5. 检查名称必须与 GitHub Checks 页面展示的名称完全一致。

Required Check	建议	作用
Go Unit Tests	必须	单元测试失败则禁止合并。
GLM-5.2 Security Audit	当前项目建议必须	安全审计未通过则禁止自动合并。
Lint / Build	按需	未来可追加，不需要修改 auto-merge.yml。

5.6 是否勾选 “Require branches to be up to date before merging”

这是 Strict 与 Loose Required Checks 的主要区别。

模式	设置	适用场景
Strict	勾选	更严格：PR 必须基于最新 main 重新验证；小中型团队更推荐。
Loose	不勾选	减少重复 CI，但并发 PR 较多时可能在 merge 后暴露集成冲突。

建议

当前阶段建议先使用 Strict。后续 PR 并发量大时，可以评估 GitHub Merge Queue，而不是简单关闭所有严格校验。

5.7 额外建议

- 启用 “Block force pushes” 保护 main。
- 尽量不要给普通用户或机器人配置 Ruleset bypass。
- 如果使用安全审计作为 Required Check，确保 audit Workflow 不会在正常内部 PR 上被 skip，否则 PR 可能长期等待。

5.8 老版 Branch protection 的备用路径

如果你的 Repository UI 没有使用 Rulesets，也可以通过旧版分支保护完成类似配置：

1. GitHub → Repository → Settings。
2. 左侧 → Branches。
3. Branch protection rules → Add branch protection rule。
4. Branch name pattern 填 main。
5. 勾选 Require a pull request before merging。
6. 勾选 Require status checks to pass before merging。
7. 添加 Go Unit Tests 和现有 Security Audit Check。
8. 保存规则。

6. GitHub Actions Token 限配置

单元测试保持只读；只有 auto-merge Workflow 需要写权限。

6.1 Repository 的 Actions 设置路径

- 1. GitHub → Repository → Settings。
- 2. 左侧 → Actions。
- 3. Actions → General。
- 4. 向下找到 “Workflow permissions” 。
- 5. 推荐仓库默认权限保持尽量收敛；在具体 Workflow 内通过 permissions 显式申请最小权限。
- 6. 如果 Organization / Enterprise Policy 禁止 Workflow 获取写权限，需要由组织管理员调整上级策略。

6.2 权限分配原则

Workflow	permissions	原因
test.yml	contents: read	只需要 Checkout 和执行测试。
audit.yml	contents: read	审计通常只读取 Diff/代码；若要写 PR 评论再单独加 pull-requests: write。
auto-merge.yml	contents: write + pull-requests: write	需要修改 PR Auto-merge 状态并最终允许合并。

不要全局放大权限

不要为了 auto merge 把 test.yml / audit.yml 也改成 write。最小权限可以明显降低 PR 代码、第三方 Action 或供应链问题的影响面。

7. 创建 auto-merge.yml

创建文件：.github/workflows/auto-merge.yml

```
name: Enable Auto Merge

on:
  pull_request:
    branches:
      - main
    types:
      - opened
      - reopened
      - synchronize
      - ready_for_review

permissions:
  contents: write
  pull-requests: write

concurrency:
  group: auto-merge-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  enable-auto-merge:
    name: Enable Auto Merge
    runs-on: ubuntu-latest
    timeout-minutes: 5

    # Draft PR 不合并; Fork PR 默认不自动合并。
    if: >-
      ${{
        !github.event.pull_request.draft &&
        github.event.pull_request.head.repo.full_name == github.repository
      }}

    steps:
      - name: Verify GitHub CLI
        run: gh --version

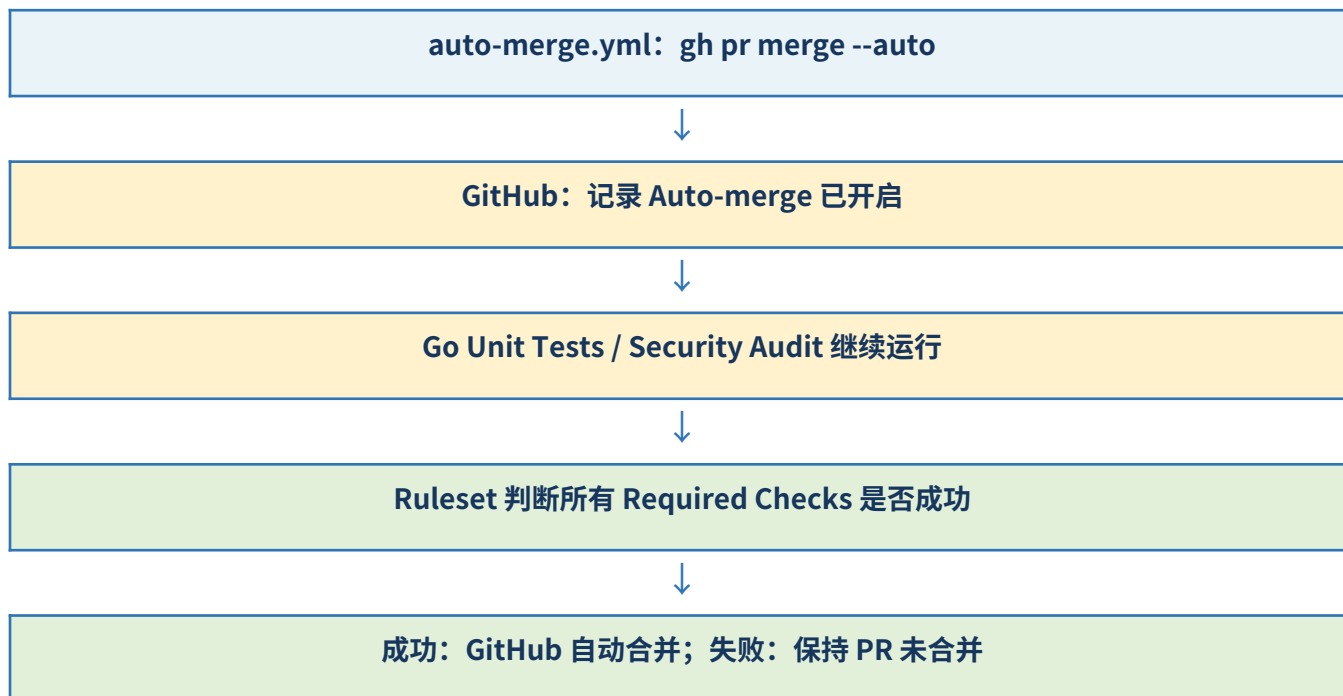
      - name: Enable GitHub auto-merge
        env:
          GH_TOKEN: ${{ github.token }}
          PR_URL: ${{ github.event.pull_request.html_url }}
          HEAD_SHA: ${{ github.event.pull_request.head.sha }}
        run: |
          set -euo pipefail

          gh pr merge \
            "${PR_URL}" \
            --auto \
            --squash \
            --match-head-commit "${HEAD_SHA}"
```

8. auto-merge.yml 的运行逻辑

8.1 为什么不需要主动等待 test.yml

auto-merge.yml 的职责不是 “等测试结束后执行 git merge”，而是提前为 PR 开启 GitHub 原生 Auto-merge。GitHub 服务端会持续观察 Ruleset：只有 Required Checks / Reviews 全部满足时才真正合并。



8.2 为什么使用 --match-head-commit

该参数要求 PR 当前 HEAD 必须等于 Workflow 触发时拿到的 HEAD_SHA。它可以降低 “针对旧 commit 开启自动化，却在 PR 已经变更后误操作新状态” 的风险。PR 有新 commit 时，synchronize 事件会重新触发 Workflow。

8.3 为什么推荐 --squash

Feature PR 常包含多个开发过程 commit。Squash Merge 可以把一个 PR 压成一个 main commit，主干历史更容易阅读。若团队需要保留完整 commit，可把 --squash 改为 --merge 或 --rebase。

8.4 Draft PR

auto-merge.yml 通过 if 条件跳过 Draft PR。准备合并时可执行：

```
gh pr ready
```

ready_for_review 事件会再次触发 auto-merge.yml。

8.5 Fork PR

示例配置只自动处理当前仓库自身分支创建的 PR。外部 Fork PR 默认不进入自动合并，避免把不可信外部贡献直接纳入无人值守合并链路。

重要：不要把 auto-merge.yml 本身设成 Required Check

真正应该 Required 的是测试、构建和安全检查。auto-merge.yml 只是开关自动合并。如果把它也作为 Required Check，会增加无意义的耦合和排障复杂度。

9. 完整运行时流程

9.1 开发者操作

```
git checkout -b feature/payment

# 开发...

git add .
git commit -m "feat(payment): implement payment service"
git push -u origin feature/payment

gh pr create --base main --fill
```

9.2 GitHub 自动执行

- PR opened → test.yml 启动 “Go Unit Tests”。
- PR opened → audit.yml（如有）启动 “GLM-5.2 Security Audit”。
- PR opened → auto-merge.yml 执行 gh pr merge --auto，为 PR 开启 Auto-merge。
- Ruleset 检查 Required Checks。
- 任何 Required Check 失败：PR 不合并。
- 开发者 push 修复 commit → pull_request:synchronize → test/audit/auto-merge 全部针对最新 commit 重跑。
- 所有 Required Checks 成功，且其他 Ruleset 条件满足 → GitHub 自动 Squash Merge 到 main。

9.3 失败与修复状态机

场景	GitHub 行为	开发者动作
Unit Test FAIL	Required Check 红色，禁止合并	修代码 → push → 自动重跑
Security Audit FAIL	安全检查不通过，禁止合并	修漏洞 → push → 自动重审

PR 新 commit	旧任务取消/过期，新 SHA 重新检查	无需手工重启
Draft PR	auto-merge job 跳过	gh pr ready
Merge conflict	GitHub 无法合并	更新分支解决冲突
Required Review 未通过	Auto-merge 持续等待	完成 Review/Approval

10. main 合并后的再次测试：生产注意事项

test.yml 中保留了 push: main，可以用于普通 push、人工 merge 或由正常用户/GitHub 服务触发的 main 更新。但如果某个事件明确由 Repository 的 GITHUB_TOKEN 触发，GitHub 会抑制大多数由该 token 产生的后续 Workflow Run，以避免递归。

不要把 Post-Merge Test 当成自动合并前置条件

真正的 Merge Gate 必须是 PR HEAD 上已经通过的 Required Checks。main 合并后的 test 只能作为额外验证，不能替代 Pre-Merge Required Check。

10.1 如果你必须保证“自动合并后 push main 一定触发另一个 Workflow”

不要依赖 Repository GITHUB_TOKEN 产生的事件链。可改用 GitHub App installation token 或合适范围的 Personal Access Token 来执行需要继续触发事件的自动化动作。生产环境优先推荐 GitHub App，因为权限和生命周期更容易控制。

10.2 当前阶段的推荐

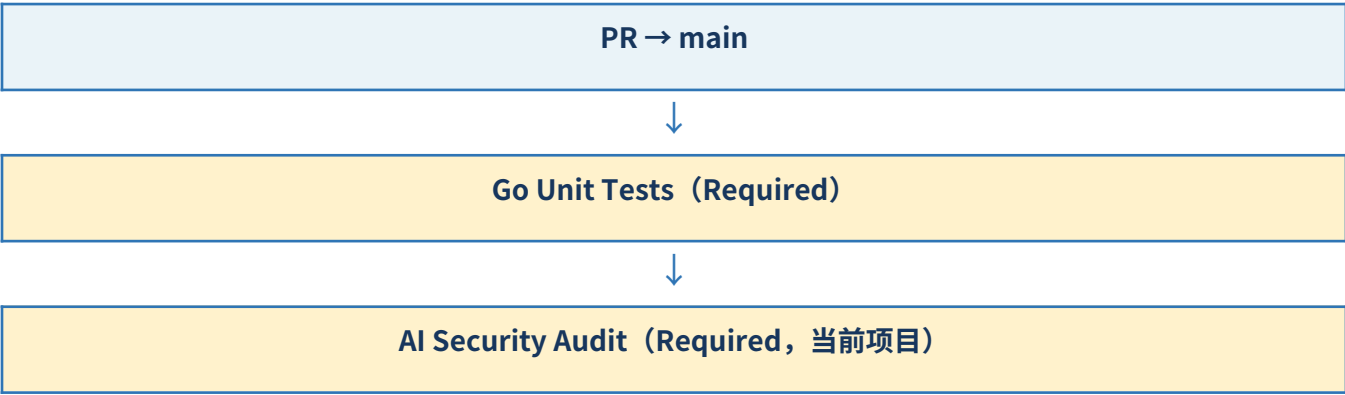
- 把 Go Unit Tests（以及现有 AI Security Audit）设置成 PR 的 Required Checks。
- 自动合并前必须全绿，这是主安全边界。
- push main 的单元测试可以保留作为额外观测，但不要依赖它决定 PR 是否安全可合并。

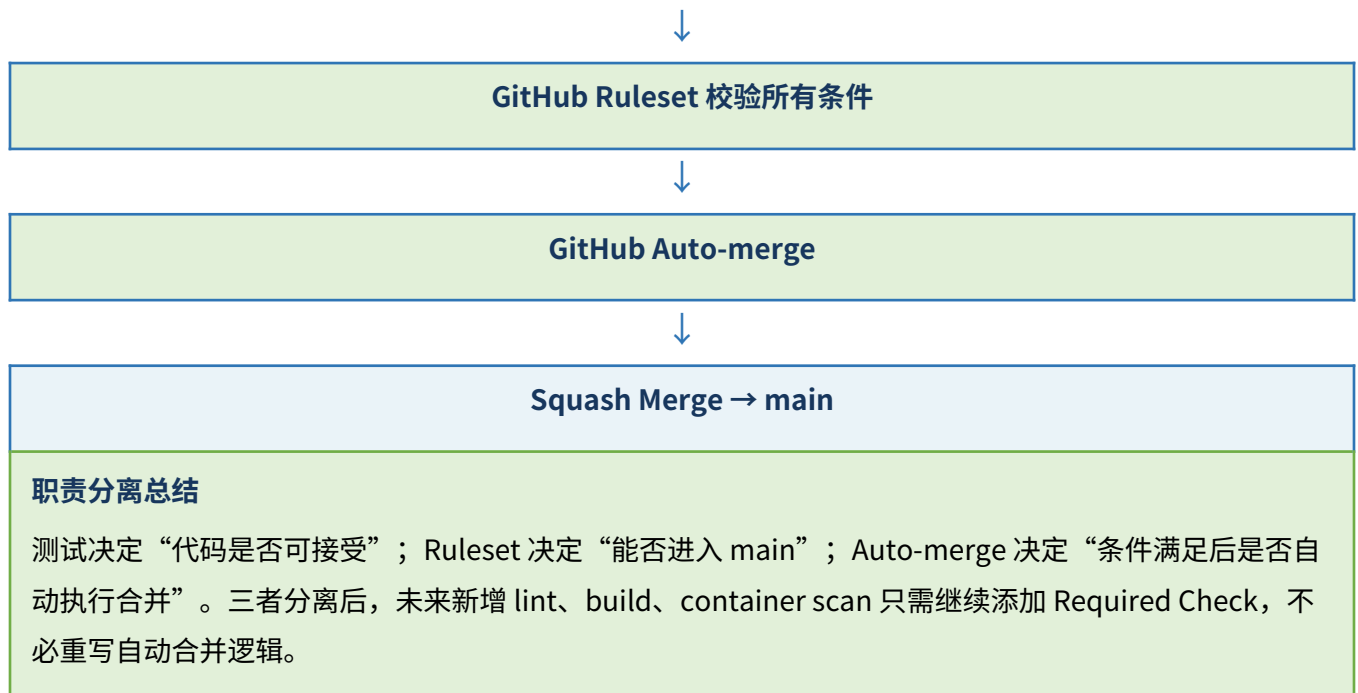
11. 上线配置检查清单

类别	检查项
Workflow	已创建 .github/workflows/test.yml
Workflow	Go Unit Tests 在真实 PR 上成功运行至少一次
Workflow	已创建 .github/workflows/auto-merge.yml

Auto-merge	Repository → Settings → General → Pull Requests → Allow auto-merge 已开启
Merge method	Repository 已允许 Squash merge（如果 auto-merge.yml 使用 --squash）
Ruleset	Repository → Settings → Rules → Rulesets → main-protection 为 Active
Ruleset	Target branch 包含 main
Ruleset	Require a pull request before merging 已开启
Ruleset	Require status checks to pass before merging 已开启
Required Check	Go Unit Tests 已加入
Required Check	GLM-5.2 Security Audit（当前项目使用时）已加入
Permissions	test.yml 仅 contents: read
Permissions	auto-merge.yml 仅申请 contents: write + pull-requests: write
Security	Fork PR 默认不自动合并
Security	Draft PR 默认不自动合并
Test	创建测试 PR，故意让 go test 失败，确认无法合并
Test	修复测试后 push，确认 synchronize 自动重跑
Test	所有 Required Checks 通过后确认 PR 自动进入 main

12. 推荐的生产最终形态





12.1 后续可扩展

- 增加 golangci-lint / go vet / govulncheck / gosec。
- 把 Build / Container Scan 继续加入 Required Checks。
- PR 并发量增大时评估 Merge Queue。
- 需要自动化动作继续触发其他 Workflow 时，用 GitHub App Token 替代 GITHUB_TOKEN。

附录 A：GitHub 官方配置系

主题	GitHub UI / 命令
开启 Auto-merge	Repository → Settings → General → Pull Requests → Allow auto-merge
创建 Ruleset	Repository → Settings → Rules → Rulesets → New ruleset → New branch ruleset
Required Checks	Ruleset → Branch protections → Require status checks to pass before merging
Actions Token 默认设置	Repository → Settings → Actions → General → Workflow permissions
PR 自动合并命令	gh pr merge <PR> --auto --squash --match-head-commit <SHA>
查看 PR 检查	gh pr checks 或 GitHub PR 页面 → Checks

参考依据：GitHub Docs 的 Auto-merge、Rulesets、Required Status Checks、GITHUB_TOKEN 与 GitHub CLI 官方说明。GitHub UI 文案可能随版本/组织策略略有差异，但配置层级和核心概念保持一致。